UTM UNIVERSITI TEKNOLOGI MALAYSIA | Faculty of Computing

Semester 01 2025/2026

Subject : Database Programming (SECP3623)
Section : Section 1-2
Task : Project I
Due : 28th November 2025 (10%)

Name	Matric No.
MUHAMMAD ADAM BIN RAZALI	A23CS0116
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
IMAN ABADI BIN MOHD NIZWAN	A23CS0084
MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112

Presentation Link: https://youtu.be/pHbNJKPfUws

Instructions

Each group must submit **one (1)** project only.

Your submission must include:

1. **Project Report** in **PDF format** (named as LeaderName_GroupNum_ProjectI.pdf).
2. The **demo video URL** (YouTube or Google Drive link) **inside the report**.
    - Make sure the link is **working**, **publicly accessible**, and does **not require login**.
3. Include all SQL script files (.sql) by attaching them directly, compressing them into one zip folder, or providing a valid OneDrive sharing link inside the report.

TABLE OF CONTENTS

1.0 Introduction

1.1 Background and Objective

This project aims to focus on the design and the implementation of a database system for a university Course Registration System. A course registration system is a process or online platform that allows students to enroll in courses for an academic term. It facilitates student-led course selection and online registration and often includes features for managing course schedules, checking prerequisites, and making payments. However, this system only covers course selection and checking prerequisites. The main objective of this project is to demonstrate how relational databases like SQL handle relationships between students, lecturers, and course registration as well as mandatory prerequisites through database management systems.

Furthermore, the project will also explore database performance optimization through the implementation of indexing to speed up data retrieval and query execution, which is mandatory when handling large amounts of growing data records. The use of constraints such as foreign keys is enforced to prevent data errors and maintain data integrity. Using MySQL Workbench, a system was developed that handles main course registration operations. This includes registering students into programs (such as Data Engineering and Software Engineering), assigning lecturers to specific courses, and processing course enrollments. The project also stresses the use of SQL for data definition, data manipulation, and advanced data retrieval to simulate functional systems.

Through this project, we were able to create a structured database solution. The system is designed to solve specific data management problems, including:

- **Prerequisite Checking**: Ensuring students cannot register for certain course that requires the completion of the foundational subjects first
- **Capacity Monitoring**: Manage class size for each courses to ensure the number of registration does not exceed the maximum capacity and allows for immediate replanning
- **Performance Optimization:** Implementing BTREE indexes for numeric data (such as intake years) and TEXT indexes for searching. This analysis demonstrates how indexing reduces the "rows examined" during complex queries.

1.2 Team Members and Assigned Roles

1. MUHAMMAD ADAM BIN RAZALI A23CS0116
    - In charge of making the introduction as well as the objective of the system in the project report.
    - Develop a data retrieval SQL queries (Part C) on question grouping and filtering groups, numeric and string functions and conditional logic.
    - Explain change in key usage on using BTREE index.
2. MUHAMMAD AFIQ DANIAL BIN ROZAIDIE A23CS0117
    - In charge of making each team member assigned roles as well as conclusion inside the project report.
    - Develop a data retrieval SQL queries (Part C) on question subqueries, set operations and joins.
    - Explain change in key usage on using TEXT index.
3. IMAN ABADI BIN MOHD NIZWAN A23CS0084
    - In charge of creating database and tables (Part A) and provide the records for all the tables (Part B)
    - Create a TEXT index for part D
    - Analyse query plans before and after index TEXT
4. MOHAMED ALIF FATHI BIN ABDUL LATIF A23CS0112
    - Develop a data retrieval SQL queries (Part C) on question filtering, sorting and aggregation.
    - Create a BTREE index for part D
    - Analyse query plans before and after index BTREE.

2.0 SQL Implementation Summary

2.1 SQL File Summaries

SQL File 1: course_registration_ddl_dml.sql

The file mainly contains DDL statements that are used to create databases and tables with proper data types and constraints. It also showcases the use of ALTER and DROP TABLE statements. The file also covers the use of DML statements such as data insertion using the INSERT statement to populate the database with at least 10 records per table. The file covers the use of UPDATE and DELETE statements using the WHERE statement with appropriate conditions to facilitate proper updates and deletion.

CREATE Statement Example

```
-- Database creation with the name course_registration
CREATE DATABASE course_registration;
```

```
-- Students table creation
CREATE TABLE students (
    student_id VARCHAR(20) PRIMARY KEY,
    student_fname VARCHAR(20) NOT NULL,
    student_lname VARCHAR(20) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    program ENUM('Data Engineering', 'Bioinformatics', 'Software Engineering', 'Networks and Security', 'Graphics and Multimedia') NOT NULL,
    intake_year INT NOT NULL,
    status VARCHAR(20) DEFAULT 'ACTIVE'
);
```

Above is the example of using the CREATE statement to create a database and database tables. The name of the database is course_registration. There are a total of 5 tables created, which are as follows:

Table Name	Explanation
students	Stores student details and academic program
lecturers	Holds lecturer information and department
courses	Defines courses, assigned lecturers, and prerequisites
registrations	Records student enrolment per semester
prerequisites	Handles prerequisite relationships and was planned to be used in the self-join task but was later dropped when adding prerequisites in courses.

INSERT Statement Example

```
INSERT INTO lecturers (lecturer_id, lecturer_fname, lecturer_lname, email, department) VALUES
('L23AC001', 'Farid', 'Hamzah', 'farid.hamzah@university.edu', 'Applied Computing'),
('L23AC002', 'Alicia', 'Tan', 'alicia.tan@university.edu', 'Applied Computing'),
('L23CS001', 'Rahman', 'Yusof', 'rahman.yusof@university.edu', 'Computer Science'),
('L23CS002', 'Nadia', 'Amin', 'nadia.amin@university.edu', 'Computer Science'),
('L23EC001', 'Suhana', 'Ariffin', 'suhana.ariffin@university.edu', 'Emergent Computing'),
('L23SE001', 'Daniel', 'Lim', 'daniel.lim@university.edu', 'Software Engineering'),
('L24AC001', 'Harith', 'Samsul', 'harith.samsul@university.edu', 'Applied Computing'),
('L24CS001', 'Melissa', 'Teh', 'melissa.teh@university.edu', 'Computer Science'),
('L24CS002', 'Arif', 'Kamal', 'arif.kamal@university.edu', 'Computer Science'),
('L24EC001', 'Jasmin', 'Ong', 'jasmin.ong@university.edu', 'Emergent Computing');
```

Above is an example of using the INSERT statement to populate the database with a minimum of 10 records. Below are the number of records for each table.

Table Name	No. of Records
students	19 (20 before DELETE statement)
lecturers	10
courses	10
registrations	19

SQL File 2: *course_registration_dql_question_1_2_3.sql*

This SQL file mainly demonstrates the use of common DQL operations which are Filtering, Sorting, and Aggregation. Filtering is done using WHERE with operators like AND, OR, NOT, BETWEEN, LIKE, and IS NULL to select specific records like filter students by intake year, program, or names starting with a letter. Sorting is performed using ORDER BY and LIMIT to arrange data like sorting students by their most recently registered courses or showing the top 5 newest students. Aggregation uses functions such as COUNT, SUM, and AVG to summarize data like count students per program or compute total credit hours. Overall, the file shows how DQL retrieves, filters, orders, and summarizes data in the database.

SQL File 3: *course_registration_dql_question_4_5_6.sql*

This sql file operates to perform three distinct categories of data analysis and manipulation. Firstly, it uses aggregation techniques with **GROUP BY** and **HAVING** clauses to filter and summarize data, specifically identifying departments with more than two lecturers, courses with multiple student registrations, and students enrolled in more than two subjects. This script also implements data. formatting, such as formatting student names to **UPPERCASE** and checking their **LENGTH**, **CONCAT** student names, extracting intake years using **SUBSTR**, and performing mathematical calculations like **ROUND** and **TRUNCATE**. Finally, it employs conditional logic using **CASE** statements to create new categorical groupings.

SQL File 4: *course_registration_dql_question_7_8_9.sql*

This SQL file focuses on retrieving information from the course_registration database using different query techniques. It finds specific students and courses using subqueries, combines results with set operations like UNION and NOT EXIST, and links related tables using several types of joins such as NATURAL, INNER, LEFT OUTER and SELF JOIN.

SQL File 5: *course_registration_btree_index.sql*

This file focuses on evaluating the performance impact of creating a B-Tree index. The goal is to measure how indexing improves query speed when filtering data. To achieve this, the query performance was first tested without any index. A simple SELECT statement using the BETWEEN operator on reg_date was executed while MySQL profiling was enabled. After recording the execution time, an index named idx_reg_date was created on the reg_date field. The same query was then executed again and the profiling output was collected for comparison. Below is the SQL statement used to create the B-Tree index.

```
CREATE INDEX idx_reg_date
ON registrations (reg_date);
```

SQL File 6: *course_registration_text_index.sql*

This file focuses on the implementation of an index called TEXT used for text searching. The SELECT statement was tested without any index before any index was applied. There is a query that uses LIKE with wildcard (%...%). Then a FULLTEXT index on the course_desc column was created, and the performance was tested after so that it can be compared with the before the result. Below is the creation of a full-text index named idx_course-desc on *t*he *course_desc* field from the *courses* table.

```
-- Step 2: Create a FULLTEXT index on the course_desc column
CREATE FULLTEXT INDEX idx_course_descp ON courses(course_desc);
```

2.2 Constraints Used

1. **PRIMARY KEY**

The primary key was used so that every record is uniquely identifiable and prevents duplicates. The primary key for each table is

Table Name	Field
students	student_id
lecturers	lecturer_id
courses	course_id
registrations	reg_id

2. **FOREIGN KEY**

A foreign key is used to maintain referential integrity between tables.

Foreign Key	Explanation
courses(lecturer_id) references lecturers(lecturer_id)	Courses must be linked to an existing lecturer
registrations(student_id) references students(student_id)	Registrations must refer to valid students and valid courses
registrations(course_id) references courses (course_id)	

3. **NOT NULL**

The NOT NULL constraint is applied to fields such as *names, email and program* that prevent incomplete or invalid records by ensuring critical fields must always contain values.

UNIQUE

Ensures that all values in a column (or set of columns) are unique.

Fields	Explanation
students (email)	Prevents duplicate email addresses
lecturers (email)	
courses(course_code)	Ensures course codes remain institution-wide unique
registrations (student_id, course_id, semester)	Prevents duplicate course registration within the same semester for the same student

4. **DEFAULT**

Provides a default value for a column when no value is explicitly specified during an INSERT operation.

- students (status) = 'ACTIVE'
- courses (capacity) = 40

5. **ENUM**

ENUM restricts a column or variable to a predefined set of values. ENUM was used to restrict degree programs and departments to predefined categories, preventing invalid entries.

Fields	Explanation
students (program ENUM(...))	Only programs in the enumerated list, which are Data Engineering, Bioinformatics, Software Engineering, Networks and Security, Graphics and Multimedia are allowed in the *programs* field.
students (department ENUM(...))	Only departments in the Applied Computing, Computer Science, Emergent Computing, Software Engineering are allowed in the *department* field.

6. **CHECK**

Enforces a specific condition that all values in a column must satisfy.

Fields	Explanation
CHECK(courses (capacity) > 0)	Course capacity cannot be negative
CHECK(courses (credit_hour) > 0)	Course credit hour cannot be negative

3.0 Query Demonstrations

- Database Creation: Create database and tables

```
-- Database creation with the name course_registration
CREATE DATABASE course_registration;
```

```
-- Students table creation
CREATE TABLE students (
    student_id VARCHAR(20) PRIMARY KEY,
    student_fname VARCHAR(20) NOT NULL,
    student_lname VARCHAR(20) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    program ENUM('Data Engineering', 'Bioinformatics', 'Software Engineering', 'Networks and Security', 'Graphics and Multimedia') NOT NULL,
    intake_year INT NOT NULL,
    status VARCHAR(20) DEFAULT 'ACTIVE'
);
```

The queries above are the example of using the CREATE statement to create a database named *course_registration* and a database table named *students*.

- Database Creation: Apply constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT

```
-- Course table creation
CREATE TABLE courses (
    course_id VARCHAR(8) PRIMARY KEY,
    course_code VARCHAR(20) UNIQUE NOT NULL,
    course_title VARCHAR(150) NOT NULL,
    credit_hours INT NOT NULL CHECK(credit_hours > 0),
    capacity INT NOT NULL DEFAULT 40 CHECK(capacity > 0),
    lecturer_id VARCHAR(20),
    prerequisite_id VARCHAR(8) DEFAULT NULL,
    course_desc TEXT,
    FOREIGN KEY (lecturer_id) REFERENCES lecturers(lecturer_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);
```

The above query is an example that covers all of the constraints, which are PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT. The *courses* table stores all course information, where *course_id* is the PRIMARY KEY to uniquely identify each course, and *course_code* is marked UNIQUE to ensure no two courses share the same official code. The *course_title* is set to NOT NULL because every course must have a name, while *credit_hours* and capacity use CHECK constraints to enforce valid positive values, with *capacity* also having a DEFAULT of 40 to automatically set a standard class size if none is provided. The *lecturer_id* is linked to the lecturers table through a FOREIGN KEY, ensuring each course references a valid lecturer and automatically updating or setting it to NULL if the lecturer changes or is removed. *prerequisite_id* is optional (NULL allowed) for courses

without prerequisites, and course_desc uses the TEXT type to store long descriptions. These constraints collectively ensure data accuracy, consistency, and integrity within the database.

- Database Creation: ALTER TABLE

```
-- Alter table modifying course_desc column
ALTER TABLE courses
MODIFY COLUMN course_desc TEXT NOT NULL;
```

The query above is an example of using the ALTER TABLE statement where *course_desc* from the *courses* table is modified to have the NOT NULL constraints.

- Database Creation: DROP TABLE IF EXISTS

```
-- Drop prerequisites table
DROP TABLE IF EXISTS prerequisites;
```

The query above drops the table named "*prerequisites*" if it exists in the database.

- Database Manipulation: Insert at least 10 records per table using INSERT.

```
INSERT INTO lecturers (lecturer_id, lecturer_fname, lecturer_lname, email, department) VALUES
('L23AC001', 'Farid', 'Hamzah', 'farid.hamzah@university.edu', 'Applied Computing'),
('L23AC002', 'Alicia', 'Tan', 'alicia.tan@university.edu', 'Applied Computing'),
('L23CS001', 'Rahman', 'Yusof', 'rahman.yusof@university.edu', 'Computer Science'),
('L23CS002', 'Nadia', 'Amin', 'nadia.amin@university.edu', 'Computer Science'),
('L23EC001', 'Suhana', 'Ariffin', 'suhana.ariffin@university.edu', 'Emergent Computing'),
('L23SE001', 'Daniel', 'Lim', 'daniel.lim@university.edu', 'Software Engineering'),
('L24AC001', 'Harith', 'Samsul', 'harith.samsul@university.edu', 'Applied Computing'),
('L24CS001', 'Melissa', 'Teh', 'melissa.teh@university.edu', 'Computer Science'),
('L24CS002', 'Arif', 'Kamal', 'arif.kamal@university.edu', 'Computer Science'),
('L24EC001', 'Jasmin', 'Ong', 'jasmin.ong@university.edu', 'Emergent Computing');
```

Above is an example of using the INSERT statement to populate the database with a minimum of 10 records.

- Database Manipulation: UPDATE with WHERE

```
-- Update lecturers table where the lecturer_id is L24CS002
UPDATE lecturers
SET department = 'Emergent Computing'
WHERE lecturer_id = 'L24CS002';
```

The above query updates the lecturers table where it sets the department to ‘Emergent Computing’ where the *lecturer_id* is L24CS002.

- Database Manipulation: DELETE with WHERE

```
-- Delete students from students table where student_id is A25CS0020
DELETE FROM students
WHERE student_id = 'A25CS0020';
```

The above query deletes record(s) from the table named *students* where the *student_id* is A25CS0020.

- Data Retrieval Question 1: Filtering

Filtering is used to retrieve only the rows that meet specific conditions. The following keywords allow precise control over which data is returned.

i. WHERE and AND

WHERE used to define the main condition for filtering rows and AND combines multiple conditions where all conditions must be true.

```
-- Find students from the 2023 intake who are in Software Engineering
SELECT student_id, student_fname, student_lname, program
FROM students
WHERE intake_year = 2023 AND program = 'Software Engineering';
```

student_id	student_fname	student_lname	program
A23CS0002	Afiq	Danial	Software Engineering
A23CS0006	Sarah	Aqilah	Software Engineering
NULL	NULL	NULL	NULL

This query returns only students who joined in 2023 and are enrolled in Software Engineering.

ii. OR

OR used returns rows where at least one condition is true.

```
-- Students who are in either Data Engineering OR Bioinformatics
SELECT student_id, student_fname, student_lname, program
FROM students
WHERE program = 'Data Engineering' OR program = 'Bioinformatics';
```

student_id	student_fname	student_lname	program
A23CS0001	Iman	Nizwan	Data Engineering
A23CS0005	Hazim	Roslan	Bioinformatics
A23CS0007	Daniel	Hakim	Data Engineering
A24CS0009	Mika	Hafiz	Bioinformatics
A24CS0012	Faris	Azlan	Data Engineering
A24CS0015	Amani	Najwa	Bioinformatics
A25CS0017	Sofea	Nurin	Data Engineering
NULL	NULL	NULL	NULL

This query retrieves students from either Data Engineering or Bioinformatics.

iii. NOT

NOT used to exclude rows that meet a condition.

```sql
-- Courses that are NOT taught by the Applied Computing department
SELECT course_code, course_title, lecturer_id
FROM courses
WHERE lecturer_id NOT IN (
    SELECT lecturer_id
    FROM lecturers
    WHERE department = 'Applied Computing'
);
```

course_code	course_title	lecturer_id
CS1013	Introduction to Computer Science	L23CS001
CS1023	Programming Fundamentals	L23CS002
CS2013	Object-Oriented Programming	L24CS001
CS2023	Data Structures and Algorithms	L24CS002
EC3013	Machine Learning Fundamentals	L23EC001
SE1013	Introduction to Software Engineering	L23SE001
CS1033	Database Systems	L24EC001

This query shows all courses not taught by lecturers from the Applied Computing department.

iv. BETWEEN

BETWEEN used to check if a value falls within a specific range.

```sql
-- Students with intake year between 2023 and 2024
SELECT student_id, student_fname, student_lname, intake_year
FROM students
WHERE intake_year BETWEEN 2023 AND 2024;
```

student_id	student_fname	student_lname	intake_year
A23CS0001	Iman	Nizwan	2023
A23CS0002	Afiq	Danial	2023
A23CS0003	Alif	Fathi	2023
A23CS0004	Irfan	Shaqir	2023
A23CS0005	Hazim	Roslan	2023
A23CS0006	Sarah	Aqilah	2023
A23CS0007	Daniel	Hakim	2023
A23CS0008	Alya	Suhana	2023
A24CS0009	Mika	Hafiz	2024
A24CS0010	Syafiq	Amir	2024

This query returns students from the intake year 2023 or 2024.

v. LIKE

LIKE performs pattern matching on text fields.

```
-- Students whose first name starts with 'A'
SELECT student_id, student_fname, student_lname
FROM students
WHERE student_fname LIKE 'A%';
```

student_id	student_fname	student_lname
A23CS0002	Afiq	Danial
A23CS0003	Alif	Fathi
A23CS0008	Alya	Suhana
A24CS0015	Amani	Najwa
NULL	NULL	NULL

This query returns students whose names start with letter A.

vi. NULL

NULL used to check for missing or empty values.

```
-- Courses that do NOT have prerequisites
SELECT course_id, course_code, course_title
FROM courses
WHERE prerequisite_id IS NULL;
```

course_id	course_code	course_title
C0000001	CS1013	Introduction to Computer Science
C0000005	AC1013	Discrete Mathematics
C0000008	SE1013	Introduction to Software Engineering
NULL	NULL	NULL

This query returns courses that do not have any prerequisite.

- Data Retrieval Question 2: Sorting

Sorting organizes query output so the results are easier to analyze.

i. ORDER BY

ORDER BY used to specify how to sort the results. ASC is for sorting from smallest to largest and DESC is for sorting from largest to smallest.

```
-- Most recent registrations first
SELECT reg_id, student_id, course_id, reg_date
FROM registrations
ORDER BY reg_date DESC;
```

reg_id	student_id	course_id	reg_date
18	A25CS0016	C0000001	2025-09-10
19	A25CS0019	C0000005	2025-09-10
17	A24CS0010	C0000009	2025-07-16
4	A23CS0001	C0000003	2025-02-17
15	A24CS0009	C0000002	2025-01-15
16	A24CS0010	C0000008	2025-01-10
6	A23CS0002	C0000008	2024-08-16
9	A23CS0003	C0000004	2024-07-17
14	A24CS0009	C0000001	2024-07-11
8	A23CS0003	C0000002	2024-03-15

This query displays course registration records starting from the most recent date.

ii. LIMIT

LIMIT used to restrict the number of rows returned.

```
-- Top 5 newest students
SELECT student_id, student_fname, student_lname, intake_year
FROM students
ORDER BY intake_year DESC
LIMIT 5;
```

student_id	student_fname	student_lname	intake_year
A25CS0017	Sofea	Nurin	2025
A25CS0016	Rayan	Hakimi	2025
A25CS0019	Maisarah	Ismail	2025
A25CS0018	Izz	Haikal	2025
A24CS0012	Faris	Azlan	2024

This query shows the 5 newest students based on intake year.

- Data Retrieval Question 3: Aggregation

Aggregation functions used to perform calculations on groups of rows. It is often used with GROUP BY.

i. COUNT

COUNT used to count the number of rows.

```
-- Total number of students in each program
SELECT program, COUNT(*) AS total_students
FROM students
GROUP BY program;
```

program	total_students
Data Engineering	4
Software Engineering	4
Networks and Security	4
Graphics and Multimedia	4
Bioinformatics	3

This query returns how many students are in each program.

ii. SUM

SUM used to add up all numeric values.

```
-- Total credit hours a student registered in a semester
SELECT student_id, semester, SUM(credit_hours) AS total_credits
FROM registrations r
JOIN courses c ON r.course_id = c.course_id
GROUP BY student_id, semester;
```

student_id	semester	total_credits
A23CS0001	23/24-1	6
A23CS0002	23/24-1	3
A23CS0003	23/24-1	3
A23CS0004	23/24-1	3
A24CS0009	24/25-1	3
A25CS0016	25/26-1	3
A23CS0001	23/24-2	3

This query calculates the total credit hours each student registered per semester.

iii. AVG

AVG used to find the average of a numeric column.

```sql
-- Average course capacity across all courses
SELECT AVG(capacity) AS avg_capacity
FROM courses;
```

avg_capacity
36.0000

This query shows the average class capacity for all courses.

iv. MAX

MAX used to return the highest value.

```sql
-- Course with the highest capacity
SELECT course_code, course_title, capacity
FROM courses
WHERE capacity = (SELECT MAX(capacity) FROM courses);
```

course_code	course_title	capacity
SE2023	Software Design and Architecture	50

This query finds the course with the highest capacity.

v. MIN

MIN used to return the smallest value.

```sql
-- Course with the smallest number of credit hours
SELECT course_code, course_title, credit_hours
FROM courses
WHERE credit_hours = (SELECT MIN(credit_hours) FROM courses);
```

course_code	course_title	credit_hours
CS1013	Introduction to Computer Science	3
CS1023	Programming Fundamentals	3
CS2013	Object-Oriented Programming	3
CS2023	Data Structures and Algorithms	3
AC1013	Discrete Mathematics	3
AC2023	Computational Mathematics	3
EC3013	Machine Learning Fundamentals	3
SE1013	Introduction to Software Engineering	3

This query finds the course with the lowest credit hours.

- Data Retrieval Question 4: Grouping and filtering groups

```
-- Count of lecturers for each departments which exceeds 2
SELECT department, COUNT(lecturer_id) AS total_lecturer
FROM lecturers
GROUP BY department
HAVING total_lecturer > 2;
```

Output:

department	total_lecturer
Applied Computing	3
Computer Science	3
Emergent Computing	3

The query above finds the total numbers of lecturers for each of the departments available through GROUPING BY the rows in the lecturers table based on department. Then, applying HAVING filters to remove any departments that have 2 or fewer lecturers.

- Data Retrieval Question 5: Numeric and string functions

```
-- List students with only short name
SELECT
    student_id,
    UPPER(CONCAT(student_fname, ' ', student_lname)) AS full_name,
    SUBSTR(student_id, 2, 2) AS year_intake,
    LENGTH(CONCAT(student_fname, ' ', student_lname)) AS name_count
FROM students
WHERE LENGTH(CONCAT(student_fname, ' ', student_lname)) > 12;
```

Output:

student_id	full_name	year_intake	name_count
A23CS0001	IMAN NIZWAN	23	11
A23CS0002	AFIQ DANIAL	23	11
A23CS0003	ALIF FATHI	23	10
A23CS0008	ALYA SUHANA	23	11
A24CS0009	MIKA HAFIZ	24	10
A24CS0010	SYAFIQ AMIR	24	11
A24CS0012	FARIS AZLAN	24	11
A24CS0013	NADIA SOFEA	24	11

The query above demonstrates several string functions to format student data. It created student's full names by concatenating their first and last name through CONCAT which is then converted into UPPERCASE. Then the SUBSTR syntax is utilized to extract the intake year from the student id. Finally the student with names not longer than 12 characters is filtered.

```sql
-- Monitor capacity of the class for each course
SELECT course_code, capacity,
ROUND(capacity * 0.85, 2) AS warning_level_rounded,
TRUNCATE(capacity * 0.85, 2) AS warning_level_truncated
FROM courses;
```

course_code	capacity	warning_level_rounded	warning_level_truncated
CS1013	40	34.00	34.00
CS1023	30	25.50	25.50
CS2013	40	34.00	34.00

This query above shows an example of a warning signal for when the class capacity reaches a certain threshold in this case, 85% full. We used ROUND and TRUNCATE to demonstrate the difference of implementation applied for both of them and how it handles precision.

- Data Retrieval Question 6: Conditional logic

```sql
-- Classify students by seniority based on intake year
SELECT
    student_id,
    intake_year,
    CASE
        WHEN intake_year = 2023 THEN 'Senior'
        WHEN intake_year = 2024 THEN 'Junior'
        WHEN intake_year = 2025 THEN 'Freshman'
        ELSE 'Unknown'
    END AS student_year_group
FROM students;
```

Output:

A23CS0007	2023	Senior
A23CS0008	2023	Senior
A24CS0009	2024	Junior
A24CS0010	2024	Junior
A24CS0011	2024	Junior
A24CS0012	2024	Junior
A24CS0013	2024	Junior
A24CS0014	2024	Junior
A24CS0015	2024	Junior
A25CS0016	2025	Freshman

The query above is implemented to dynamically categorize intake year into seniority label which through the application of the CASE and WHEN syntax. This conditional logic splits

students from different intake years starting from 2023, 2024, and 2025 and returns it as a student_year_group.

- Data Retrieval Question 7: Subqueries

i. Single Row

```
SELECT student_id, student_fname, student_lname, email, program, intake_year
FROM students
WHERE intake_year = (
    SELECT MAX(intake_year)
    FROM students
);
```

Output:

student_id	student_fname	student_lname	email	program	intake_year
A25CS0016	Rayan	Hakimi	rayan.hakimi@cs.edu	Software Engineering	2025
A25CS0017	Sofea	Nurin	sofea.nurin@cs.edu	Data Engineering	2025
A25CS0018	Izz	Haikal	izz.haikal@cs.edu	Networks and Security	2025
A25CS0019	Maisarah	Ismail	maisarah.ismail@cs.edu	Graphics and Multimedia	2025
NULL	NULL	NULL	NULL	NULL	NULL

This query finds all students whose intake year is the most recent year. It first gets the maximum intake year from the students table, then returns every student whose intake year matches that maximum value.

ii. Multiple Row

```
SELECT lecturer_id, course_id, course_code, course_title
FROM courses
WHERE lecturer_id IN (
    SELECT lecturer_id
    FROM lecturers
    WHERE department = 'Applied Computing'
);
```

Output:

lecturer_id	course_id	course_code	course_title
L23AC001	C0000005	AC1013	Discrete Mathematics
L23AC002	C0000006	AC2023	Computational Mathematics
L24AC001	C0000009	SE2023	Software Design and Architecture
NULL	NULL	NULL	NULL

This query lists all courses taught by lecturers from the Applied Computing department. The subquery first retrieves all lecturer IDs in that department, and the main query returns any course whose lecturer_id matches one of those IDs.

iii. Correlated

```
SELECT r1.student_id, r1.course_id, r1.semester
FROM registrations r1
WHERE (
    SELECT COUNT(*)
    FROM registrations
    WHERE r1.student_id = student_id
) > 2;
```

Output:

student_id	course_id	semester
A23CS0001	C0000001	23/24-1
A23CS0001	C0000002	23/24-2
A23CS0001	C0000003	24/25-1
A23CS0001	C0000005	23/24-1
A23CS0003	C0000001	23/24-1
A23CS0003	C0000002	23/24-2
A23CS0003	C0000004	24/25-2

This query finds students who have registered for more than 2 courses. The subquery counts how many registrations each student has, and because it references r1.student_id, it is correlated. Only students whose total course count is greater than 2 are returned.

- Data Retrieval Question 8: Set Operations

i. UNION

```
SELECT cou  rse_id, course_title, credit_hours
FROM courses
WHERE credit_hours = 3
UNION
SELECT course_id, course_title, credit_hours
FROM courses
WHERE capacity > 40;
```

Output:

course_id	course_title	credit_hours
C0000001	Introduction to Computer Science	3
C0000002	Programming Fundamentals	3
C0000003	Object-Oriented Programming	3
C0000004	Data Structures and Algorithms	3
C0000005	Discrete Mathematics	3
C0000006	Computational Mathematics	3
C0000007	Machine Learning Fundamentals	3
C0000008	Introduction to Software Engineering	3
C0000009	Software Design and Architecture	3
C0000010	Database Systems	3

This query combines two sets of courses which are Courses that have 3 credit hours and Courses that have capacity greater than 40. UNION merges both results into one list and removes duplicates.

ii. NOT EXIST

```
SELECT s.student_id, CONCAT(s.student_fname,' ', s.student_lname) AS student_fullname
FROM students s
WHERE NOT EXISTS (
    SELECT 1
    FROM registrations r
    WHERE r.student_id = s.student_id
      AND r.course_id = 'C0000001'
);
```

Output:

student_id	student_fullname
A23CS0005	Hazim Roslan
A23CS0006	Sarah Aqilah
A23CS0007	Daniel Hakim
A23CS0008	Alya Suhana
A24CS0010	Syafiq Amir
A24CS0011	Hannah Zulaikha
A24CS0012	Faris Azlan
A24CS0013	Nadia Sofea
A24CS0014	Zharif Imran
A24CS0015	Amani Najwa
A25CS0017	Sofea Nurin
A25CS0018	Izz Haikal
A25CS0019	Maisarah Ismail

This query retrieves a list of students who have not enrolled in the course “Introduction to Computer Science” (C0000001). It works by checking each student in the students table against the registrations table to see if there is a corresponding record for that specific course. Using the NOT EXISTS clause, the query ensures that only students who do not have any registration for that course are included in the result.

- Data Retrieval Question 9: Joins

i. NATURAL

```
SELECT student_id, CONCAT(student_fname,' ', student_lname) AS fullname,
       course_id, semester
FROM registrations
NATURAL JOIN students;
```

Output:

student_id	fullname	course_id	semester
A23CS0001	Iman Nizwan	C0000001	23/24-1
A23CS0001	Iman Nizwan	C0000002	23/24-2
A23CS0001	Iman Nizwan	C0000003	24/25-1
A23CS0001	Iman Nizwan	C0000005	23/24-1
A23CS0002	Afiq Danial	C0000001	23/24-1
A23CS0002	Afiq Danial	C0000008	23/24-2
A23CS0003	Alif Fathi	C0000001	23/24-1
A23CS0003	Alif Fathi	C0000002	23/24-2
A23CS0003	Alif Fathi	C0000004	24/25-2
A23CS0004	Irfan Shaqir	C0000001	23/24-1
A23CS0004	Irfan Shaqir	C0000010	23/24-2
A23CS0005	Hazim Roslan	C0000005	23/24-1

This query retrieves a list of all course registrations along with the full names of the students. It uses a NATURAL JOIN between the registrations and students tables, which automatically joins the tables on the column they have in common (student_id).

ii. INNER JOIN

```
SELECT  c.course_code, c.course_title,
        UPPER(CONCAT(l.lecturer_fname,' ', l.lecturer_lname)) AS lecturer_fullname,
        l.lecturer_id
FROM courses c
INNER JOIN lecturers l
ON c.lecturer_id = l.lecturer_id;
```

Output:

course_code	course_title	lecturer_fullname	lecturer_id
CS1013	Introduction to Computer Science	RAHMAN YUSOF	L23CS001
CS1023	Programming Fundamentals	NADIA AMIN	L23CS002
CS2013	Object-Oriented Programming	MELISSA TEH	L24CS001
CS2023	Data Structures and Algorithms	ARIF KAMAL	L24CS002
AC1013	Discrete Mathematics	FARID HAMZAH	L23AC001
AC2023	Computational Mathematics	ALICIA TAN	L23AC002
EC3013	Machine Learning Fundamentals	SUHANA ARIFFIN	L23EC001
SE1013	Introduction to Software Engineering	DANIEL LIM	L23SE001
SE2023	Software Design and Architecture	HARITH SAMSUL	L24AC001
CS1033	Database Systems	JASMIN ONG	L24EC001

This query retrieves a list of courses along with the details of the lecturers teaching them. It uses an INNER JOIN between the courses and lecturers tables on the lecturer_id column, so only courses that have an assigned lecturer are included.

iii. LEFT OUTER

```
SELECT c.course_id, c.course_title, s.student_id,
       LOWER(CONCAT(s.student_fname,' ', s.student_lname)) AS fullname_student
FROM courses c
LEFT OUTER JOIN registrations r
ON c.course_id = r.course_id
LEFT OUTER JOIN students s
ON r.student_id = s.student_id;
```

Output:

course_id	course_title	student_id	fullname_student
C0000001	Introduction to Computer Science	A23CS0001	iman nizwan
C0000001	Introduction to Computer Science	A23CS0002	afiq danial
C0000001	Introduction to Computer Science	A23CS0003	alif fathi
C0000001	Introduction to Computer Science	A23CS0004	irfan shaqir
C0000001	Introduction to Computer Science	A24CS0009	mika hafiz
C0000001	Introduction to Computer Science	A25CS0016	rayan hakimi
C0000002	Programming Fundamentals	A23CS0001	iman nizwan
C0000002	Programming Fundamentals	A23CS0003	alif fathi
C0000002	Programming Fundamentals	A24CS0009	mika hafiz
C0000003	Object-Oriented Programming	A23CS0001	iman nizwan
C0000004	Data Structures and Algorithms	A23CS0003	alif fathi

This query retrieves a list of all courses along with the students registered for each course, if any. It uses LEFT OUTER JOINs to ensure that every course appears in the result, even if no students are registered. The first join links courses to registrations by course_id, and the second join links registrations to students by student_id.

iv. SELF JOIN

```
SELECT c.course_id, c.course_title, p.course_id, p.course_title
FROM courses c
LEFT JOIN courses p
    ON c.prerequisite_id = p.course_id;
```

Output:

course_id	course_title	course_id	course_title
C0000001	Introduction to Computer Science	NULL	NULL
C0000002	Programming Fundamentals	C0000001	Introduction to Computer Science
C0000003	Object-Oriented Programming	C0000002	Programming Fundamentals
C0000004	Data Structures and Algorithms	C0000003	Object-Oriented Programming
C0000005	Discrete Mathematics	NULL	NULL
C0000006	Computational Mathematics	C0000005	Discrete Mathematics
C0000007	Machine Learning Fundamentals	C0000004	Data Structures and Algorithms
C0000008	Introduction to Software Engineering	NULL	NULL
C0000009	Software Design and Architecture	C0000008	Introduction to Software Engineering
C0000010	Database Systems	C0000001	Introduction to Computer Science

This query retrieves a list of courses along with their prerequisite courses using a SELF JOIN on the courses table. The table is joined to itself with aliases c for the main course and p for the prerequisite course, linking c.prerequisite_id to p.course_id. A LEFT JOIN is used so that courses without prerequisites are still included, with the prerequisite columns showing NULL.

4.0 Optimization Section

4.1 B-tree indexing

```
-- Before index
SELECT *
FROM registrations
WHERE reg_date BETWEEN '2023-01-01' AND '2023-12-31';
```

reg_id	student_id	course_id	reg_date	semester
1	A23CS0001	C0000001	2023-09-10	23/24-1
2	A23CS0001	C0000005	2023-09-12	23/24-1
5	A23CS0002	C0000001	2023-01-11	23/24-1
7	A23CS0003	C0000001	2023-09-12	23/24-1
10	A23CS0004	C0000001	2023-09-10	23/24-1
12	A23CS0005	C0000005	2023-10-10	23/24-1

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	registrations	NULL	ALL	NULL	NULL	NULL	NULL	19	11.11	Using where

Execution time:

2	0.00182650	SELECT * FROM registrations WHER...

Based on the query above, we find registrations within a specific datetime. Without b-tree indexing, the query opens the registrations table and checks for every single row which is 19 rows to see if the reg_date satisfies the condition inside the date range. According to the profiling, this operation took 0.00182650 seconds for a full table rows scan.

```
CREATE INDEX idx_reg_date
ON registrations (reg_date);
```

We then executed the query above to create a balanced tree (B-tree) indexing for the reg_date called idx_reg_date.

```sql
-- After index
SELECT *
FROM registrations
WHERE reg_date BETWEEN '2023-01-01' AND '2023-12-31';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	registrations	NULL	range	idx_reg_date	idx_reg_date	4	NULL	6	100.00	Using index condition

Execution time:

Query_ID	Duration	Query
1	0.00039400	SHOW WARNINGS
2	0.00182650	SELECT * FROM registrations WHER...
3	0.06421000	CREATE INDEX idx_reg_date ON regi...
4	0.00040350	SELECT * FROM registrations WHER...

Based on the same query executed earlier, now that the indexing exists, it traverses the tree and follows the sorted links to find the starting point until it hits the endpoint, therefore skipping all rows that are outside of this range resulting in only 6 rows of checking. According to the "Duration" table (Query ID 4), this operation took 0.00040350 seconds. The query became approximately 4.5 times faster (reducing time from ~0.0018s to ~0.0004s).

4.2 Text indexing

Before Indexing

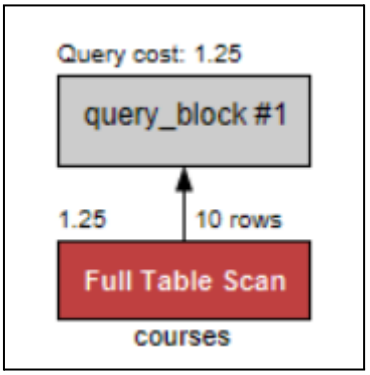
```
EXPLAIN SELECT *
FROM courses
WHERE course_desc LIKE '%classification%';

SELECT *
FROM courses
WHERE course_desc LIKE '%classification%';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	courses	NULL	ALL	NULL	NULL	NULL	NULL	10	11.11	Using where

course_id	course_code	course_title	credit_hours	capacity	lecturer_id	prerequisite_id	course_desc
C0000007	EC3013	Machine Learning Fundamentals	3	40	L23EC001	C0000004	Core ML algorithms including classification, regr...
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Execution Time:



61	0.03502325	EXPLAIN SELECT * FROM courses WHERE cour...
62	0.01006075	SELECT * FROM courses WHERE course_desc L...

Based on the query above, this query is to find course descriptions that contain the word "classification" for all the records within the course table. Without TEXT indexing, the query opens the course table and does a full table scan for every single record (10 rows) to see if the word "classification" satisfies the condition. Based on the diagram above, it shows the query cost is 1.25 which means it may use very little resources. According to the profiling, this operation took 0.03502325 seconds and 0.1006075 seconds for both queries before the TEXT indexing.

```
CREATE FULLTEXT INDEX idx_course_desc ON courses(course_desc);
```

Next, executing this SQL statement creates a full-text index named idx_course_desc on the course_desc column of the courses table. It allows fast and efficient text searches within

course descriptions, enabling queries like MATCH(...) AGAINST(...) to find words or phrases in the text.

After Indexing:

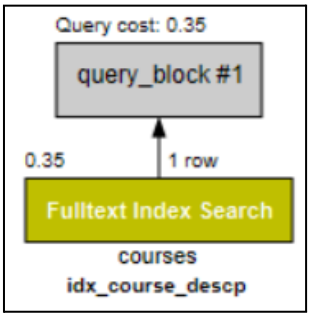
```
EXPLAIN SELECT *
FROM courses
WHERE MATCH(course_desc) AGAINST('classification' IN NATURAL LANGUAGE MODE);

SELECT *
FROM courses
WHERE MATCH(course_desc) AGAINST('classification' IN NATURAL LANGUAGE MODE);
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	courses	NULL	fulltext	idx_course_descp	idx_course_descp	0	const	1	100.00	Using where; Ft_hints: sorted

course_id	course_code	course_title	credit_hours	capacity	lecturer_id	prerequisite_id	course_desc
C0000007	EC3013	Machine Learning Fundamentals	3	40	L23EC001	C0000004	Core ML algorithms including classification, regr...
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Execution Time:



84	0.00241650	EXPLAIN EXPLAIN SELECT * FROM courses WH...
85	0.00133325	SELECT * FROM courses WHERE MATCH(course...

Based on the diagram above, this query searches for course descriptions that contain the word “classification” in the courses table. With the full-text index idx_course_desc, the query uses a Fulltext Index Search instead of scanning every row. The execution plan shows a query cost of 0.35, indicating it uses very few resources compared to full table scan. According to the profiling, the query execution time was 0.00241650 seconds for the EXPLAIN query and 0.00133325 seconds for the actual SELECT query, which is significantly faster than a full table scan without text indexing.

5.0 Team Work

Team Member	What we have learned
MUHAMMAD ADAM BIN RAZALI	In this project, I was able to learn the advanced data retrieval queries in sql which includes aggregation and grouping, string and numeric functions, as well as conditional logic using CASE. Through explaining B-tree indexing, I was able to understand database performance optimization, specifically observing how the indexing reduces execution time when scanning and analyzing tables. Overall, I am able to learn more deeply on technical skills in sql and database management systems as well as team collaboration and organization in order to divide and work on the task efficiently and precisely.
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	Through my role in this project, I gained hands-on experience in developing complex SQL queries, including subqueries, set operations, and various types of joins, which strengthened my understanding of data retrieval and relational database interactions. Additionally, I learned how indexing, particularly using a TEXT index, affects key usage and query performance by optimizing searches on text-based fields. Overall, this role enhanced both my technical SQL skills and my ability to manage and organize a collaborative project.
IMAN ABADI BIN MOHD NIZWAN	I was responsible for executing the DML and DDL questions and designing the database that is logical. I learned how different SQL constraints, such as primary keys, foreign keys, unique, check, not null, and default, help maintain data accuracy and consistency in a database. I was also responsible for handling the TEXT indexing and gained experience applying indexing and understanding query performance differences between LIKE searches and FULLTEXT indexing.
MOHAMED ALIF FATHI BIN ABDUL LATIF	For this project, I handled Part C Question 1, 2, and 3, which involved writing SQL queries using filtering, sorting, grouping, and aggregation. Through these tasks, I improved my skills in retrieving and analyzing data using different SQL functions and clauses. I also created a B-Tree index and learned how indexing improves performance by speeding up searches and reducing the amount of data scanned. Overall, the project strengthened my SQL fundamentals and gave me a clearer understanding of how databases manage and optimize queries.

6.0 Conclusion

In conclusion, this project successfully demonstrates how a well-designed relational database can support the essential operations of a university Course Registration System. By structuring data around students, lecturers, courses, prerequisites, and registrations, the system ensures accurate and consistent data handling. The implementation of foreign keys, constraints, and indexing strengthens both data integrity and performance, proving essential when managing large and growing datasets.

Throughout the project, challenges were encountered such as handling complex prerequisite checks, ensuring course capacity limits were strictly enforced, and optimizing queries for faster performance on large datasets. To address these challenges, suggestions for improvement include implementing more advanced indexing strategies, adding automated alerts for capacity management, and integrating stored procedures or triggers to handle complex validation rules more efficiently. Through this project, we also learn the importance of proper database design, how SQL's DDL, DML, and DQL features work together, and gain practical experience in building scalable, reliable, and optimized database solutions suitable for real-world academic applications.

Marking Rubric

SQL Implementation (CLO1)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Backend SQL Implementation (DDL/DML/Queries)	All SQL statements execute flawlessly; covers full range – DDL, DML, SELECT, joins, subqueries, functions.	Most SQL correct; small syntax or logic issues.	Several working statements but limited variety.	Few working statements; repetitive or basic logic.	Non-functional / incomplete SQL.	
Advanced Data Logic (Filtering, Aggregation, Subquery)	Uses complex logic (nested queries, GROUP BY, HAVING, CASE, conditions) effectively.	Correct use of aggregations with minor gaps.	Some queries too simple or misapplied.	Few aggregations or logical errors.	Not attempted.	
Indexing & Optimization	Correctly creates BTREE and TEXT indexes; shows analysis before/after; interprets performance gain.	Indexes implemented; partial analysis.	Indexing attempted without analysis.	Incomplete index usage.	Not attempted.	
Transaction Control & Front-End Logic Integration	Demonstrates atomic operations (START TRANSACTION, COMMIT, ROLLBACK); explains how it fits system workflow (front-end use case e.g. booking/purchase).	Transaction logic corrects but limited integration explanation.	Partial transaction implementation.	Minimal attempt, unclear scenario.	Missing transaction logic.	
Subtotal						

Teamwork and Collaboration (CLO3)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Team Coordination & Contribution	Clear task division; balanced participation; strong collaboration evident in report and presentation; each member contributes to SQL work.	Good teamwork and participation with minor imbalance.	Some collaboration; limited task clarity.	Minimal teamwork; dominated by few members.	No teamwork evidence; incomplete group submission.	
Subtotal						

Documentation & Presentation (CLO4)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Report Documentation	Clear and complete report including intro, SQL summary, query screenshots, optimization, transaction explanation, and reflection.	Report complete with minor formatting or clarity issues.	Adequate report but lacks details or screenshots.	Weakly structured or incomplete sections.	Missing or very poor documentation.	
Slide Quality & Visual Communication	Slides are professional, organised, visually clear, and consistent with project flow.	Well-prepared slides with minor design issues.	Slides understandable but inconsistent layout.	Poorly structured or cluttered slides.	Missing or unreadable slides.	
Verbal Presentation	Excellent delivery, confident speakers, balanced time usage, fluent explanation, and effective Q&A handling.	Good delivery; some coordination issues.	Understandable but low energy; moderate Q&A.	Disorganized delivery or weak explanations.	Unable to present or answer questions.	
Subtotal						